



INFORME FINAL

Sistema de Gestión Hotelera y Servicios para Huéspedes

BRAGADO – GRUPO 16

Integrantes:

Luz Barrientos: 1193268

Juliana Galiano: 1193349

Paloma Macchión: 1197513

Día y Turno:

Miércoles Turno mañana

Profesores:

LEON MARIA ANGELA

Entrega:

20 de Junio

ÍNDICE

1. Introducción	4
2. Descripción del problema	4
3. Objetivos del sistema	5
Objetivo General	5
Objetivos Específicos	5
4. Requisitos funcionales	6
5. Casos de uso	9
Administrador	9
Recepcionista	10
Personal Administrativo	10
Huésped/Cliente	10
6. Diagrama de clases	12
7. Diagramas de secuencia	14
Crear y confirmar una reserva	14
Check out	16
8. Descripción de la arquitectura del sistema	19
Capa de presentación	19
Capa de servicios (gestores)	19
Capa de dominio (modelo)	19
Capa de patrones de diseño	19
Capa de tipos auxiliares (enums)	20
Flujo general de la información	20
9. Patrones de diseño aplicados	20
Builder	21
Factory Method	21
Decorator	22
State	23
Strategy	24
Observer	24
10. Justificación de SOLID	25
SRP - Single Responsibility Principle	25
OCP - Open/Closed Principle	26
DIP - Dependency Inversion Principle	26
11. Justificación de GRASP	26
Controller	26
Creator	27
Low Coupling	27
High Cohesion	27
Polymorphism	28

12. Explicación de las clases principales	28
Habitacion (abstracta)	28
Reserva	28
Huesped	28
UsuarioInterno (abstracta)	28
Pago	29
Gestores (ReservaGestor, HabitacionGestor, EstadiaGestor)	29
13. Instrucciones de despliegue / ejecución del proyecto	29
14. Capturas de ejecución o ejemplos de uso	29
15. Conclusión grupal	35
16. Reflexión individual de cada integrante	35

ENLACE DEL REPOSITORIO:

<https://github.com/juligali/Tp-GestionHotelera>

1. Introducción

El presente informe final tiene como objetivo integrar y presentar el desarrollo del proyecto Bragado - Sistema de Gestión Hotelera y Servicios para Huéspedes, realizado a partir de las etapas de análisis, diseño e implementación inicial del sistema.

En una primera instancia, se realizó el análisis general de la problemática, identificando la necesidad de una cadena hotelera de centralizar la gestión de reservas, habitaciones, huéspedes, estadías, pagos y servicios adicionales. A partir de esta situación, se definieron los objetivos del sistema, el alcance funcional, los requisitos funcionales y no funcionales, los actores principales y los casos de uso más relevantes.

Luego, en la segunda etapa, se avanzó con el diseño orientado a objetos del sistema, incorporando diagramas UML, diagramas de secuencia, patrones de diseño, principios SOLID y patrones GRASP. Estas herramientas permiten organizar mejor las responsabilidades de cada clase, reducir el acoplamiento entre componentes y plantear una estructura flexible, mantenible y extensible.

El sistema propuesto busca automatizar procesos que normalmente podrían realizarse de forma manual, como la creación y confirmación de reservas, la consulta de disponibilidad, la asignación de habitaciones, el check-in, el check-out, el cálculo de costos, la aplicación de descuentos y el envío de notificaciones. De esta manera, se apunta a mejorar la organización interna del hotel, evitar errores administrativos y brindar una experiencia más eficiente tanto para el personal como para los huéspedes.

A lo largo del informe se presenta la evolución del proyecto desde su análisis inicial hasta su diseño técnico, mostrando cómo se aplicaron conceptos de programación orientada a objetos y buenas prácticas de diseño de software para construir una base sólida del sistema.

2. Descripción del problema

Una cadena hotelera necesita contar con un sistema que le permita centralizar y organizar la gestión de sus principales operaciones. Actualmente, muchas tareas relacionadas con reservas, habitaciones, huéspedes, estadías, pagos y servicios adicionales pueden realizarse de forma manual o mediante herramientas poco integradas. Esto puede generar errores en la disponibilidad de habitaciones, reservas superpuestas, demoras administrativas y dificultades para calcular correctamente el costo final de una estadía.

El problema principal se encuentra en la falta de un sistema unificado que permita administrar de manera ordenada todos los procesos del hotel. La ausencia de una solución centralizada puede afectar tanto al personal interno como a los huéspedes, ya que complica la consulta de disponibilidad, la asignación de habitaciones, el seguimiento de reservas y el registro de servicios contratados.

Por este motivo, se propone el desarrollo de BRAGADO, un sistema de gestión hotelera orientado a objetos que permita automatizar procesos frecuentes, mejorar la organización interna y reducir errores operativos. El sistema busca ofrecer una solución flexible y extensible, capaz de adaptarse a distintos tipos de habitaciones, servicios adicionales, promociones, descuentos y políticas comerciales.

3. Objetivos del sistema

Objetivo General

Desarrollar un sistema integral de gestión hotelera que permita administrar de manera centralizada las reservas, habitaciones, huéspedes, estadías y servicios asociados, optimizando los procesos operativos del hotel mediante una solución orientada a objetos, flexible, extensible y basada en buenas prácticas de diseño de software.

El sistema busca reducir errores derivados de la gestión manual, mejorar el control de disponibilidad de habitaciones, facilitar el cálculo automático de costos y brindar una experiencia más organizada tanto para el personal del hotel como para los huéspedes.

Objetivos Específicos

- **Automatizar la gestión de reservas y estadías:**
Permitir la creación, confirmación, cancelación y seguimiento de reservas, así como el inicio y cierre de estadías mediante procesos de check-in y check-out digital.
- **Reducir errores y demoras ocasionadas por procesos manuales:**
Centralizar la información del hotel para evitar inconsistencias, reservas duplicadas, demoras administrativas y problemas en la asignación de habitaciones.
- **Mejorar el control y disponibilidad de habitaciones:**
Permitir la consulta y actualización del estado de las habitaciones, identificando si se encuentran disponibles, reservadas, ocupadas, en limpieza o en mantenimiento.
- **Asignar habitaciones de manera automática:**
Buscar y asignar una habitación disponible que coincida con el tipo solicitado, la capacidad requerida y las fechas seleccionadas.
- **Gestionar distintos tipos de habitaciones de forma flexible:**
Administrar habitaciones simples, dobles, suites, familiares u otras categorías que el hotel pueda incorporar en el futuro.
- **Asignar habitaciones de manera automática:**
Buscar y asignar una habitación disponible que coincida con el tipo solicitado, la capacidad requerida y las fechas seleccionadas.
- **Gestionar servicios adicionales de forma dinámica:**
Incorporar servicios personalizados a una estadía, como desayuno, spa, estacionamiento, traslado o late check-out, permitiendo combinaciones según las necesidades del huésped.

- **Implementar un cálculo automático y adaptable de costos:**
Calcular el costo total de una estadía considerando la cantidad de noches, el tipo de habitación, los servicios adicionales, promociones y descuentos aplicables.
- **Registrar y consultar información de huéspedes:**
Mantener un registro digital de huéspedes con sus datos personales, información de contacto e historial de estadías.
- **Mejorar la experiencia del huésped:**
Brindar una atención más eficiente y personalizada mediante la gestión organizada de reservas, servicios, preferencias e historial de estadías.
- **Generar reportes básicos de ocupación:**
Permitir al personal administrativo consultar el estado general del hotel, identificando habitaciones disponibles, reservadas, ocupadas, en limpieza o en mantenimiento.
- **Facilitar la incorporación de nuevas políticas comerciales:**
Diseñar el sistema de manera extensible para permitir futuras promociones, descuentos por temporada, beneficios para clientes frecuentes o nuevas reglas de negocio.
- **Favorecer futuras integraciones externas:**
Preparar la arquitectura del sistema para una posible integración con plataformas de reservas online, sistemas de pago, agencias externas o programas de fidelización.

4. Requisitos funcionales

Los siguientes requisitos describen las funcionalidades efectivamente implementadas en el sistema.

RF01 - Registro y consulta de huéspedes

El sistema permite registrar huéspedes (el alta se realiza automáticamente al crear una reserva, a partir de su nombre, email y teléfono) y consultar el listado de huéspedes junto con su historial de reservas. El listado de huéspedes está disponible para el Administrador y el Personal Administrativo.

RF02 - Registro y consulta de habitaciones

El sistema permite registrar nuevas habitaciones (a cargo del Administrador) y consultar el listado de habitaciones con sus características: número, tipo, capacidad, precio por noche, estado y descripción.

RF03 - Consulta de disponibilidad por tipo

El sistema permite consultar las habitaciones disponibles filtrando por tipo de habitación. Se consideran disponibles las habitaciones cuyo estado actual es "Disponible".

RF04 - Gestión de reservas

El sistema permite crear, confirmar, modificar y cancelar reservas.

- El huésped/cliente puede crear, consultar, modificar y cancelar sus propias reservas desde el portal de cliente.
- El Recepcionista y el Administrador pueden crear, confirmar y cancelar reservas.

RF05 - Gestión del estado de la reserva

El sistema administra el ciclo de vida de una reserva mediante los estados Pendiente, Confirmada, Cancelada y Finalizada, controlando qué operaciones son válidas en cada estado (patrón State).

RF06 - Registro de estadías (check-in y check-out)

El sistema permite registrar el check-in (creación de la estadía a partir de una reserva confirmada) y el check-out (cierre de la estadía y generación del pago).

RF07 - Asignación de habitaciones

El sistema asigna a cada reserva una habitación disponible del tipo solicitado, actualizando su estado a "Reservada".

RF08 - Incorporación de servicios adicionales

El sistema permite agregar servicios adicionales a una estadía (desayuno, spa y estacionamiento), que se combinan dinámicamente y suman su costo a la estadía (patrón Decorator).

RF09 - Cálculo automático de costos

El sistema calcula automáticamente el costo total de la estadía considerando la cantidad de noches, el precio del tipo de habitación, los servicios adicionales y el descuento aplicado.

RF10 - Aplicación de descuentos

El sistema permite aplicar distintas políticas de descuento al confirmar la reserva: sin descuento, descuento por temporada (15%) y descuento por cliente frecuente (10%) (patrón Strategy).

RF11 - Extensibilidad de tipos de habitación

El sistema permite incorporar nuevos tipos de habitación sin modificar el funcionamiento existente, mediante la creación de una nueva clase de habitación y su fábrica correspondiente (patrón Factory Method).

RF12 - Gestión de estados de habitaciones

El sistema administra los estados de las habitaciones: disponible, reservada, ocupada, limpieza y fuera de servicio. El Administrador y el Recepcionista pueden asignar manualmente los estados de limpieza y fuera de servicio.

RF13 - Historial de reservas del huésped

El sistema permite consultar el historial de reservas de cada huésped, con el estado de cada una.

RF14 - Registro de pagos (simulado)

El sistema registra el pago asociado a una estadía al realizar el check-out, indicando monto y método de pago.

RF15 - Emisión de comprobantes (simulada)

El sistema genera un comprobante de pago con el detalle de la transacción (ID, monto, fecha y método).

RF16 - Notificaciones (simuladas)

El sistema notifica al huésped por email y SMS ante eventos de la reserva (confirmación, cancelación y finalización), mediante el patrón Observer.

RF17 - Reportes de ocupación

El sistema genera reportes con información del estado general del hotel: total de habitaciones, disponibles, reservadas, ocupadas, porcentaje de ocupación, reservas activas, pagos registrados e ingresos simulados.

RF18 - Autenticación de usuarios internos

El sistema permite a los usuarios internos (Administrador, Recepcionista y Personal Administrativo) autenticarse mediante email y contraseña, accediendo únicamente a las funcionalidades correspondientes a su rol.

RF19 - Control de acceso por roles

El sistema restringe las operaciones disponibles según el rol del usuario, tanto en el menú/interfaz como en la validación de cada gestor.

RF20 - Filtrado de reservas y disponibilidad

El sistema permite filtrar las reservas de un cliente por su email y las habitaciones por tipo, tanto en el portal de cliente como en la vista interna.

RF21 - Disponibilidad por fechas y capacidad

El sistema prevé que cada habitación mantenga una agenda de reservas y que la disponibilidad se calcule comprobando que no exista solapamiento de fechas, además de filtrar por capacidad.

Posibles mejoras a futuro

Se incluyen funcionalidades previstas en el alcance del diseño de las etapas anteriores que no fueron implementadas en esta etapa, o que se implementaron de forma parcial.

- Bloqueo temporal de habitaciones. Se prevé incorporar un bloqueo provisional de la habitación durante el proceso de reserva, para evitar que dos usuarios reserven la misma habitación simultáneamente.

- Gestión de usuarios internos. Actualmente los usuarios internos están predefinidos en el sistema. Como mejora, se prevé que el Administrador pueda crear, modificar y eliminar usuarios internos, asignándoles un rol.
- Registro y autenticación de huéspedes. Se prevé un registro con contraseña y un inicio de sesión real para el portal del cliente (actualmente el acceso se realiza solo mediante el email, sin contraseña).
- Gestión del perfil del huésped. Se prevé permitir al huésped modificar sus datos personales desde su cuenta (actualmente el perfil es de solo lectura).
- Promociones comerciales. Además de los descuentos actuales, se prevé incorporar promociones como entidad propia (paquetes, beneficios por temporada, etc.).
- Recordatorios automáticos. Sumar a las notificaciones existentes el envío de recordatorios previos a la fecha de la reserva.
- Reportes ampliados. Incorporar reportes adicionales, como un historial de estadías por huésped.
- Emisión de facturas y pagos reales. Reemplazar el comprobante simulado por la emisión de facturas formales e integrar medios de pago reales.
- Integraciones externas. Preparar la integración con plataformas de reservas online, sistemas de pago y programas de fidelización.
- Persistencia de datos. Incorporar una base de datos para que la información persista entre ejecuciones (actualmente los datos se mantienen solo en memoria).

5. Casos de uso

Administrador

- Crear habitación.
- Cambiar estado de habitación.
- Crear/confirmar/cancelar reserva.
- Check-in.
- Check-out.
- Agregar servicios.
- Listar habitaciones/reservas/estadías/pagos.
- Listar huéspedes.
- Ver reportes.

Recepcionista

- Crear/confirmar/cancelar reserva.
- Check-in.
- Check-out.
- Agregar servicios.
- Cambiar estado de habitación.
- Listar habitaciones/reservas/estadías/pagos.

Personal Administrativo

- Listar huéspedes.
- Ver reportes.
- Listar reservas/estadías/pagos.

Huésped/Cliente

- Consultar disponibilidad.
- Crear reserva.
- Consultar sus reservas.
- Modificar su reserva.
- Cancelar su reserva.
- Ver su perfil.

<https://tinyurl.com/msuaur3d>

Este diagrama representa los casos de uso del sistema Bragado, es decir, las funcionalidades visibles desde afuera y los permisos que tiene cada actor para utilizarlas.

Los actores del diagrama coinciden con los roles definidos en el sistema:

- Administrador
- Recepcionista
- Personal Administrativo
- Huésped / Cliente

El **Administrador** es el actor con mayor nivel de permisos dentro del sistema.

Puede gestionar habitaciones, administrar reservas, realizar check-in y check-out, y administrar servicios.

Además, es el único actor que puede crear habitaciones, ya que esta acción forma parte de la configuración y administración general del hotel.

El **Recepcionista** cumple un rol operativo dentro del sistema.

Tiene casi las mismas capacidades funcionales que el administrador en cuanto a la operación diaria del hotel: puede gestionar reservas, realizar check-in y check-out, administrar servicios y cambiar el estado de las habitaciones.

Sin embargo, no puede crear habitaciones, ya que esa responsabilidad queda reservada únicamente para el administrador.

El **Personal Administrativo** tiene un rol principalmente de consulta, control y apoyo.

Puede buscar reservas, consultar disponibilidad y cancelar reservas.

Sin embargo, no puede crear ni confirmar reservas, y tampoco opera estadias mediante check-in o check-out.

Esto refleja que su participación está orientada al seguimiento y control de la información, más que a la operación directa del alojamiento.

El **Huésped o Cliente** interactúa con el sistema desde el portal.

Puede consultar disponibilidad, crear reservas para su propia cuenta, consultar sus reservas, modificarlas o cancelarlas.

Este actor no puede confirmar reservas ni aplicar descuentos, ya que esas acciones corresponden al personal interno del hotel.

Además, sus permisos están limitados a sus propias reservas, por lo que no puede modificar ni cancelar reservas de otros huéspedes.

Las relaciones **<<extend>>** representan la secuencia lógica del negocio.

En este caso:

- Confirmar reserva extiende a crear reserva.
- Check-in extiende a confirmar reserva.
- Check-out extiende a check-in.

Esto indica que cada acción depende de una etapa anterior. Primero debe existir una reserva creada, luego esa reserva puede confirmarse, después puede realizarse el check-in y, finalmente, el check-out.

Un detalle importante, es que las restricciones sobre acciones propias del huésped no se basan únicamente en el rol.

Para modificar o cancelar una reserva propia, el sistema realiza una validación adicional mediante `validarReservaDelHuesped`.

Esta validación comprueba que la reserva pertenezca efectivamente al huésped que intenta operarla. De esta forma, el sistema evita que un huésped pueda modificar o cancelar reservas que no le corresponden.

6. Diagrama de clases

<https://tinyurl.com/ytm4j34d>

Este diagrama representa la estructura estática del sistema BRAGADO: las clases, sus atributos y métodos, y las relaciones entre ellas. Está organizado en bloques que coinciden con los paquetes del proyecto y con los patrones de diseño aplicados.

Usuarios

En la parte de usuarios, UsuarioInterno es una clase abstracta que centraliza los datos comunes: id, nombre, email, contraseña y rol, además de la lógica de login.

De esta clase heredan:

- Administrador
- Recepcionista
- PersonalAdministrativo
- Huesped

Cada subclase fija su propio Rol en el constructor, lo que permite aplicar control de acceso según el tipo de usuario.

Además, Administrador agrega una lista de usuarios que gestiona, mientras que Huesped se distingue porque mantiene su propia lista de reservas mediante los métodos agregarReserva, getReservas y consultarHistorial, además de tener un teléfono asociado.

Esto refleja el principio de alta cohesión, ya que cada tipo de usuario concentra únicamente las responsabilidades que le corresponden.

Habitaciones + Factory Method

La clase Habitacion es abstracta y define los atributos y comportamientos comunes de todas las habitaciones, como el estado, la disponibilidad y el cambio de estado.

Además, deja como métodos abstractos:

- getPrecioPorNoche
- getDescripcion

Estos métodos son implementados por sus subclases:

- HabitacionSimple
- HabitacionDoble
- HabitacionSuite

Cada una define sus propios valores y características.

La creación de habitaciones no se realiza con new directamente, sino a través de la jerarquía HabitacionFactory y sus tres fábricas concretas. Estas fábricas están conectadas con Habitacion mediante una dependencia de creación.

Esto aplica el patrón Factory Method y cumple con el principio Open/Closed, ya que se pueden agregar nuevos tipos de habitaciones sin modificar el código existente.

Reserva y patrones aplicados

La clase Reserva es el núcleo del dominio y concentra varios patrones de diseño de comportamiento y creación.

State

El patrón State se aplica mediante la interfaz o clase EstadoReserva y sus cuatro estados concretos.

La reserva delega su comportamiento al estado actual. Esta relación se representa por composición, ya que el estado forma parte intrínseca de la reserva y no tiene sentido sin ella.

Strategy

El patrón Strategy se aplica mediante EstrategiaDescuento y sus tres implementaciones.

El cálculo del descuento se encapsula en estrategias intercambiables. La relación con Reserva se representa por agregación, porque la estrategia puede cambiarse mediante cambiarEstrategia y existe de forma independiente.

Observer

El patrón Observer se aplica mediante ObservadorReserva, junto con sus implementaciones de notificación por email y SMS.

La reserva mantiene una lista de observadores a los que avisa cuando cambia de estado.

Builder

El patrón Builder se aplica con ReservaBuilder, que permite construir una reserva paso a paso y validar los datos antes de instanciarla.

Esto evita crear reservas incompletas o inválidas.

Además, Reserva se asocia con Habitacion y Huesped por agregación.

Todo esto ilustra el principio Dependency Inversion Principle, ya que Reserva depende de abstracciones como EstadoReserva, EstrategiaDescuento y ObservadorReserva, y no de clases concretas.

Estadía + Decorator

La clase Estadía representa la ocupación efectiva de una habitación y se encarga de manejar el check-in y el check-out.

Para los servicios adicionales se aplica el patrón Decorator.

La interfaz ComponenteEstadia define el costo y la descripción. Luego, EstadíaBase funciona como componente base, envolviendo la habitación y la cantidad de noches.

A partir de eso, ServicioDecorator actúa como clase abstracta para los decoradores concretos:

- DesayunoDecorator
- SpaDecorator
- EstacionamientoDecorator

Cada decorador suma costo y descripción a la estadía.

La clase Estadía guarda los servicios elegidos en un conjunto de TipoAmenity y reconstruye la cadena de decoradores al agregar o quitar amenities.

Finalmente, al realizar el check-out, la estadía genera un Pago.

Gestores / Controllers

ReservaGestor, HabitacionGestor y EstadíaGestor actúan como controladores GRASP: reciben las solicitudes, validan permisos por rol y coordinan las operaciones, pero delegan el comportamiento específico en las clases del dominio. Además de coordinar, cada gestor mantiene en memoria las entidades que administra (por ejemplo, ReservaGestor guarda la lista de reservas, HabitacionGestor la de habitaciones y EstadíaGestor las de estadías y pagos); por eso en el diagrama estas se modelan como agregaciones y no como simples dependencias. También aplican Creator, ya que crean los objetos cuyos datos coordinan: ReservaGestor construye reservas apoyándose en ReservaBuilder, y EstadíaGestor genera el Pago al hacer el check-out. La relación con HabitacionFactory, en cambio, sí es una dependencia, porque la fábrica se recibe como parámetro y no se almacena.

7. Diagramas de secuencia

Crear y confirmar una reserva

<https://tinyurl.com/y9w24yuc>

Este diagrama representa la interacción entre objetos a lo largo del tiempo para uno de los flujos más importantes del sistema BRAGADO: la creación y confirmación de una reserva.

Además, este flujo integra tres patrones de diseño trabajando en conjunto:

- Builder
- State
- Observer

Etapas de creación de la reserva

El flujo comienza cuando el Recepcionista solicita la creación de una reserva llamando al método `crearReserva` de `ReservaGestor`.

En primer lugar, el gestor valida los permisos del usuario según su rol. Esto asegura que solo los usuarios autorizados puedan realizar esta operación.

Luego, `ReservaGestor` utiliza `ReservaBuilder` para construir la reserva paso a paso, cargando los datos necesarios:

- Huésped.
- Habitación.
- Fechas de la reserva.
- Estrategia de descuento.

Por defecto, la estrategia de descuento utilizada es `SinDescuento`.

Una vez cargados todos los datos, el gestor llama al método `build()`. Este método valida que la información sea correcta y completa antes de crear la instancia de `Reserva`.

La reserva se crea inicialmente con el estado `EstadoPendiente`, por lo que nace en estado `PENDIENTE`.

Finalmente, el gestor registra la reserva dentro de la lista de reservas del huésped y devuelve la reserva al recepcionista.

Etapas de confirmación de la reserva

Luego, el Recepcionista solicita la confirmación de la reserva llamando al método `confirmarReserva`, indicando también la estrategia de descuento que se desea aplicar.

Nuevamente, `ReservaGestor` valida los permisos del usuario para comprobar que tenga autorización para confirmar reservas.

Después, el gestor aplica la estrategia de descuento seleccionada mediante el método `cambiarEstrategia`.

A continuación, invoca el método `confirmar()` sobre la reserva.

En esta parte interviene el patrón `State`. La reserva no decide directamente cómo cambiar de estado, sino que delega ese comportamiento en su estado actual.

Como la reserva se encuentra en EstadoPendiente, se ejecuta la lógica de EstadoPendiente.confirmar(), que permite realizar la transición hacia EstadoConfirmada mediante el método setEstado.

Cuando el estado cambia efectivamente, se activa el patrón Observer.

La reserva notifica a sus observadores mediante el método actualizar. Estos observadores simulan el envío de notificaciones, por ejemplo por email o SMS.

Finalmente, ReservaGestor devuelve la reserva al recepcionista en estado CONFIRMADA.

El diagrama deja en evidencia una correcta separación de responsabilidades dentro del sistema.

Cada clase cumple una función específica:

- ReservaGestor coordina el flujo y valida permisos.
- ReservaBuilder construye la reserva paso a paso.
- EstadoReserva decide si una transición de estado es válida.
- Reserva mantiene su estado y notifica los cambios.
- ObservadorReserva se encarga de comunicar las notificaciones.

De esta forma, ninguna clase concentra toda la lógica del proceso. El diseño queda más ordenado, mantenible y alineado con los principios de responsabilidad única y bajo acoplamiento.

Check out

<https://tinyurl.com/y7ubyp2x>

Este diagrama representa el flujo de cierre de una estadía dentro del sistema Bragado. En esta operación se combinan varias responsabilidades importantes: el cálculo de costos, la aplicación de servicios adicionales, el cambio de estado de la reserva y la generación del pago correspondiente.

Además, el diagrama muestra cómo trabajan en conjunto distintos patrones de diseño:

- Decorator
- Strategy
- State
- Controller / Gestores

Inicio del check-out

El flujo comienza cuando el Recepcionista solicita realizar el check-out de una estadía a través de Estadía Gestor.

En primer lugar, el gestor valida los permisos del usuario según su rol. Esto asegura que solo un usuario autorizado pueda cerrar una estadía.

Una vez validado el permiso, EstadiaGestor delega la operación en la clase Estadia, que es la encargada de manejar la lógica propia del cierre de la estadía.

Cálculo del costo total

Dentro de Estadia se realiza el cálculo del costo total, que se compone de dos partes principales.

Costo de los servicios adicionales

Por un lado, se calcula el costo de los servicios adicionales seleccionados por el huésped.

Este cálculo se realiza recorriendo la cadena de decoradores mediante el método `ComponenteEstadia.getCostoTotal()`.

Acá se aplica el patrón Decorator, ya que cada servicio adicional funciona como un decorador que envuelve al componente anterior y suma su propio costo.

Por ejemplo:

- `DesayunoDecorator` suma el costo del desayuno.
- `SpaDecorator` suma el costo del spa.
- `EstacionamientoDecorator` suma el costo del estacionamiento.

De esta forma, el sistema puede agregar servicios sin modificar la clase base de la estadía.

Costo de la habitación con descuento

Por otro lado, se calcula el costo de la habitación utilizando el método `reserva.calcularCostoTotal()`.

En esta parte interviene el patrón Strategy, ya que el cálculo puede variar según la estrategia de descuento aplicada a la reserva.

Por ejemplo, la reserva puede utilizar:

- `SinDescuento`
- Una estrategia de descuento específica
- Otra estrategia definida para una promoción o condición particular

Esto permite cambiar la forma de calcular el descuento sin modificar la lógica principal de la reserva.

Generación del pago

Una vez obtenidos los dos valores (el costo de los servicios adicionales y el costo de la habitación con descuento) la estadía arma el costo final.

Con ese total, se crea el objeto Pago.

Luego, el pago se registra mediante el método registrarPago, dejando constancia de la operación realizada.

Finalización de la reserva

Después de registrar el pago, la estadía finaliza la reserva mediante el método reserva.finalizar().

En este punto vuelve a intervenir el patrón State, ya que la reserva delega el cambio de estado en su estado actual.

Si la transición es válida, la reserva pasa al estado FINALIZADA.

Además, como parte del cierre de la estadía, la habitación queda disponible nuevamente para futuras reservas.

Generación del comprobante

Una vez finalizada la reserva y registrado el pago, el sistema genera el comprobante correspondiente mediante el método generarComprobante.

Finalmente, Estadia devuelve el pago generado a EstadiaGestor.

El gestor lo guarda y lo retorna al recepcionista como resultado de la operación.

Conclusión del diagrama

Este diagrama muestra cómo varios patrones colaboran dentro de una misma operación de negocio.

Cada patrón cumple una función específica:

- Decorator permite sumar servicios adicionales al costo de la estadía.
- Strategy permite aplicar distintas formas de descuento sobre la reserva.
- State controla la transición de la reserva hacia el estado finalizado.
- EstadiaGestor actúa como controlador, validando permisos y coordinando la operación.

En conjunto, este diseño permite mantener una buena separación de responsabilidades, bajo acoplamiento y una estructura más flexible para futuras modificaciones.

8. Descripción de la arquitectura del sistema

El sistema Bragado está desarrollado en Java siguiendo el paradigma de programación orientada a objetos, y se organiza en una arquitectura por capas basada en la separación de responsabilidades. Cada capa se corresponde con un conjunto de paquetes y cumple una función específica dentro del sistema, lo que facilita el mantenimiento, la comprensión del código y la incorporación de nuevas funcionalidades.

Capa de presentación

Es la encargada de la interacción con el usuario. El sistema ofrece dos puntos de entrada:

- Una interfaz gráfica desarrollada en JavaFX (gui/HotelApp y gui/GuiLauncher, con estilos en hotel-theme.css).
- Una interfaz de consola (Main).

Ambas cumplen el mismo rol: recibir las acciones del usuario, mostrar la información y delegar las operaciones a la capa de servicios. La mayor parte de la lógica de negocio reside en los gestores y en el modelo. No obstante, en esta etapa la capa de presentación todavía conserva parte de la lógica de apoyo - como la búsqueda de disponibilidad con detección de cruces de fechas, el cálculo de los totales con amenities y la elaboración de los reportes de ocupación e ingresos -, que en una evolución futura debería migrar a la capa de servicios para reforzar la separación de responsabilidades.

Capa de servicios (gestores)

Está formada por los gestores ReservaGestor, HabitacionGestor y EstadiaGestor. Actúan como controladores (patrón Controller de GRASP): reciben las solicitudes de la capa de presentación, validan los permisos según el rol del usuario y coordinan las operaciones sobre las entidades del dominio. Esta capa concentra la coordinación, pero delega el comportamiento específico en las clases del modelo.

Capa de dominio (modelo)

Contiene las entidades principales del negocio y su lógica: Reserva, Estadia, Habitacion (y sus subclases), Huesped, UsuarioInterno (y sus subclases) y Pago. Aquí reside el comportamiento propio de cada entidad, como el cálculo de costos, el ciclo de vida de una reserva o la generación de un comprobante. En este paquete también se define el tipo enumerado TipoAmenity (con nombre y precio de cada servicio adicional), utilizado por la estadía para gestionar los amenities.

Capa de patrones de diseño

Organizada en el paquete patrones, agrupa las implementaciones de los patrones aplicados, separadas según su categoría:

- Creacionales (builder, factory).
- Estructurales (decorator).
- Comportamiento (state, strategy, observer).

Estos patrones se integran con las entidades del dominio (por ejemplo, una Reserva usa un EstadoReserva, una EstrategiaDescuento y una lista de ObservadorReserva), aportando flexibilidad y extensibilidad sin acoplar el modelo a implementaciones concretas.

Capa de tipos auxiliares (enums)

El paquete enums define los tipos enumerados utilizados de forma transversal: TipoHabitacion, EstadoHabitacion y Rol.

Flujo general de la información

El flujo típico parte de la capa de presentación (GUI o consola), que envía la solicitud a la capa de servicios. El gestor correspondiente valida los permisos y coordina la operación, apoyándose en la capa de patrones para crear o configurar los objetos (por ejemplo, construir una reserva con el Builder o crear una habitación con la Factory). Finalmente, la operación se materializa sobre las entidades de la capa de dominio, que ejecutan su propia lógica y devuelven el resultado a la capa de presentación para mostrarlo al usuario.

Esta organización en capas, combinada con el uso de patrones de diseño y los principios SOLID, permite mantener un bajo acoplamiento entre componentes y una alta cohesión dentro de cada clase, dando como resultado un sistema ordenado, flexible y preparado para crecer.

Nota sobre persistencia: en esta etapa el sistema mantiene los datos en memoria durante la ejecución, principalmente en listas y estructuras dentro de los gestores y, en algunos casos, en estructuras auxiliares de la capa de presentación (por ejemplo, el registro de pagos y totales finales en la consola). La incorporación de una capa de persistencia con base de datos figura entre las mejoras a futuro.

9. Patrones de diseño aplicados

En el desarrollo del sistema BRAGADO se aplicaron distintos patrones de diseño GoF con el objetivo de lograr una estructura más flexible, organizada y fácil de mantener.

Los patrones aplicados permiten separar responsabilidades, evitar código repetido, reducir el acoplamiento entre clases y facilitar la incorporación de nuevas funcionalidades en el futuro.

Los patrones implementados fueron:

- Builder como patrón creacional.
- Factory Method como patrón creacional.
- Decorator como patrón estructural.
- State como patrón de comportamiento.
- Strategy como patrón de comportamiento.
- Observer como patrón de comportamiento.

Builder

Problemática detectada

El patrón Builder se aplicó para la creación de reservas.

Una reserva necesita varios datos para construirse correctamente, como el huésped, la habitación, las fechas de ingreso y egreso, y la estrategia de descuento.

Si todos estos datos se enviaran directamente por medio de un constructor tradicional, el código sería más difícil de leer, más propenso a errores y menos claro al momento de crear una reserva. Además, se correría el riesgo de generar objetos incompletos o mal configurados.

Solución con el patrón

Para resolver esta problemática se creó la clase ReservaBuilder, encargada de construir una reserva paso a paso.

El builder permite ir cargando los datos necesarios de la reserva de forma ordenada:

- }Huésped.
- Habitación.
- Fecha de ingreso.
- Fecha de egreso.
- Estrategia de descuento.

Una vez cargados los datos, se llama al método build(), que valida la información antes de crear la instancia final de Reserva.

De esta manera, la responsabilidad de construir correctamente una reserva queda separada de la clase que solicita su creación.

Ventaja obtenida

La principal ventaja del patrón Builder es que permite crear objetos complejos de manera más clara y controlada.

En el sistema BRAGADO, esto ayuda a evitar reservas incompletas o mal armadas, mejora la legibilidad del código y facilita la incorporación de nuevos atributos en el futuro sin modificar demasiado la estructura existente.

Factory Method

Problemática detectada

El patrón Factory Method se aplicó para la creación de habitaciones.

El sistema trabaja con distintos tipos de habitaciones, como habitación simple, doble y suite.

Cada una tiene características propias, como precio, capacidad y descripción.

Si cada parte del sistema instanciara directamente las clases concretas usando new, el código quedaría muy acoplado a cada tipo específico de habitación. Esto haría más difícil agregar nuevos tipos en el futuro, como una habitación familiar o premium.

Solución con el patrón

Para solucionar esto, la creación de habitaciones se delega a una jerarquía de fábricas. Se utiliza una clase o interfaz HabitaciónFactory, junto con fábricas concretas para cada tipo de habitación:

- HabitaciónSimpleFactory
- HabitaciónDobleFactory
- HabitaciónSuiteFactory

Cada fábrica concreta se encarga de crear el tipo de habitación correspondiente.

De esta manera, el sistema no necesita conocer directamente cómo se instancia cada clase concreta, sino que delega esa responsabilidad en la fábrica correspondiente.

Ventaja obtenida

La ventaja principal es que el sistema queda más desacoplado de las clases concretas de habitaciones.

Si en el futuro se quiere agregar un nuevo tipo de habitación, solo se debe crear una nueva clase de habitación y su fábrica correspondiente, sin modificar el funcionamiento general del sistema.

Esto favorece el principio Open/Closed, ya que el sistema queda abierto a la extensión, pero cerrado a la modificación.

Decorator

Problemática detectada

El patrón Decorator se aplicó para agregar servicios adicionales a una estadía.

Una estadía puede tener distintos servicios, como desayuno, spa y estacionamiento. Estos servicios pueden combinarse de muchas formas diferentes.

Si se creara una clase para cada combinación posible, por ejemplo una estadía con desayuno, una estadía con desayuno y spa, una estadía con spa y estacionamiento, etc., el sistema tendría demasiadas clases y sería difícil de mantener.

Solución con el patrón

Para resolver esta problemática se implementó el patrón Decorator.

La interfaz ComponenteEstadia define las operaciones principales relacionadas con el costo y la descripción de una estadía.

La clase EstadiaBase representa la estadía base, mientras que ServicioDecorator funciona como decorador abstracto.

A partir de esa estructura, se crean decoradores concretos para los servicios adicionales:

- DesayunoDecorator

- SpaDecorator
- EstacionamientoDecorator

Cada decorador envuelve al componente anterior y suma su propio costo y descripción. De esta manera, los servicios adicionales se agregan como capas sobre la estadía base.

Ventaja obtenida

La ventaja principal del patrón Decorator es que permite combinar servicios de manera dinámica sin modificar la clase principal de estadía.

En BRAGADO, esto permite agregar o quitar servicios adicionales de forma flexible, evitando crear una clase distinta para cada combinación posible.

Además, facilita la incorporación de nuevos servicios en el futuro, ya que solo se debería crear un nuevo decorador.

State

Problemática detectada

El patrón State se aplicó para representar el ciclo de vida de una reserva.

Una reserva puede estar en distintos estados, como pendiente, confirmada, cancelada o finalizada. Cada estado permite o restringe ciertas acciones.

Por ejemplo, no es lo mismo confirmar una reserva pendiente que intentar confirmar una reserva ya cancelada o finalizada.

Si toda esta lógica se resolviera dentro de la clase Reserva mediante muchos condicionales, el código sería más largo, más difícil de leer y más complejo de mantener.

Solución con el patrón

Para solucionar esto, cada estado de la reserva se representa mediante una clase específica.

Se utiliza una abstracción EstadoReserva y distintas implementaciones concretas, como:

- EstadoPendiente
- EstadoConfirmada
- EstadoCancelada
- EstadoFinalizada

La clase Reserva mantiene una referencia a su estado actual y delega en ese estado las operaciones correspondientes, como confirmar, cancelar o finalizar.

Por ejemplo, cuando se llama al método confirmar(), la reserva no decide directamente qué hacer, sino que delega esa acción en el estado actual.

Ventaja obtenida

La ventaja principal del patrón State es que evita cargar a la clase Reserva con muchos condicionales.

Cada estado concentra su propio comportamiento, haciendo que el código sea más ordenado, mantenible y fácil de extender.

Además, permite controlar de forma más clara qué operaciones son válidas según el estado actual de la reserva.

Strategy

Problemática detectada

El patrón Strategy se aplicó para el cálculo de descuentos.

El sistema puede aplicar distintas políticas de descuento, como no aplicar descuento, aplicar un descuento por temporada o aplicar un descuento para clientes frecuentes.

Si estas reglas estuvieran escritas directamente dentro de la clase Reserva, habría que modificar esa clase cada vez que se agregara o cambiara una política de descuento.

Esto aumentaría el acoplamiento y haría que la clase Reserva tuviera más responsabilidades de las necesarias.

Solución con el patrón

Para resolver esta problemática, cada política de descuento se encapsula en una clase diferente.

Se utiliza una abstracción EstrategiaDescuento, con distintas implementaciones concretas, como:

- SinDescuento
- DescuentoTemporada
- DescuentoClienteFrecuente

La clase Reserva utiliza una estrategia de descuento para calcular el costo total, pero no depende directamente de una implementación concreta.

Además, la estrategia puede cambiarse mediante el método cambiarEstrategia.

Ventaja obtenida

La ventaja principal del patrón Strategy es que permite cambiar la forma de calcular descuentos sin modificar la clase Reserva.

Esto hace que el sistema sea más flexible y extensible.

Si en el futuro se quiere agregar una nueva promoción o política comercial, solo se debe crear una nueva estrategia de descuento, sin afectar la lógica principal de la reserva.

Observer

Problemática detectada

El patrón Observer se aplicó para el sistema de notificaciones.

Cuando ocurre un evento importante en una reserva, como una confirmación, cancelación o finalización, el sistema debe poder notificar al huésped.

Estas notificaciones pueden realizarse por distintos medios, como email o SMS.

Si la clase Reserva dependiera directamente de cada canal de notificación, quedaría muy acoplada a implementaciones concretas y sería más difícil agregar nuevos medios en el futuro.

Solución con el patrón

Para resolver esta problemática, se implementó el patrón Observer.

La clase Reserva mantiene una lista de observadores que implementan la abstracción ObservadorReserva.

Entre los observadores concretos se encuentran las notificaciones por:

- Email.
- SMS.

Cuando la reserva cambia de estado o sucede un evento importante, la reserva notifica a sus observadores mediante el método actualizar.

De esta manera, la reserva no necesita saber exactamente cómo se envía cada notificación, solo avisa a los observadores registrados.

Ventaja obtenida

La principal ventaja del patrón Observer es que desacopla la lógica de la reserva de los canales de notificación.

Esto permite agregar nuevos medios de comunicación en el futuro, como WhatsApp o notificaciones push, sin modificar la lógica principal de Reserva.

Además, mejora la separación de responsabilidades: la reserva gestiona su estado y los observadores se encargan de comunicar los cambios.

10. Justificación de SOLID

En el sistema Bragado se aplicaron principios SOLID para mejorar la organización del código, facilitar el mantenimiento y permitir que el sistema pueda crecer sin generar modificaciones innecesarias en clases ya existentes.

SRP - Single Responsibility Principle

El principio de responsabilidad única indica que cada clase debe tener una única responsabilidad o motivo de cambio.

En el sistema, este principio se aplica porque cada clase cumple una función específica. Por ejemplo, Reserva se encarga de representar y gestionar el ciclo de vida de una reserva, Habitación administra los datos y estado de una habitación, Pago registra la información relacionada con una transacción, y los gestores coordinan operaciones del sistema.

Esto evita que una misma clase concentre demasiadas tareas y permite que el código sea más claro, ordenado y fácil de modificar.

OCP - Open/Closed Principle

El principio abierto/cerrado establece que el sistema debe estar abierto a la extensión, pero cerrado a la modificación.

Este principio se cumple porque el diseño permite agregar nuevas funcionalidades sin tener que modificar las clases ya existentes. Por ejemplo, se pueden agregar nuevos tipos de habitaciones mediante Factory Method, nuevos servicios adicionales mediante Decorator, nuevas estrategias de descuento mediante Strategy y nuevos canales de notificación mediante Observer.

Gracias a esto, el sistema puede adaptarse a cambios futuros sin afectar el funcionamiento que ya fue desarrollado.

DIP - Dependency Inversion Principle

El principio de inversión de dependencias indica que las clases principales no deben depender de implementaciones concretas, sino de abstracciones.

En el proyecto, este principio se aplica porque varias clases trabajan con interfaces. Por ejemplo, Reserva depende de EstadoReserva y no de un estado concreto específico. También trabaja con EstrategiaDescuento, sin depender directamente de una política puntual de descuento.

Esto reduce el acoplamiento y permite reemplazar o agregar nuevas implementaciones sin modificar las clases principales del sistema.

11. Justificación de GRASP

Los patrones GRASP se aplicaron para asignar correctamente las responsabilidades dentro del sistema. Estos patrones ayudan a decidir qué clase debe encargarse de cada tarea, favoreciendo un diseño más claro, mantenible y coherente con la programación orientada a objetos.

Controller

El patrón Controller se aplica mediante los gestores del sistema, como ReservaGestor, HabitacionGestor y EstadiaGestor.

Estas clases reciben las solicitudes principales del sistema y coordinan las operaciones necesarias. Por ejemplo, ReservaGestor puede coordinar la creación de una reserva, la consulta de disponibilidad, la construcción del objeto reserva y su confirmación.

De esta forma, los gestores funcionan como intermediarios entre las acciones del usuario y las clases del dominio, evitando que las entidades principales queden sobrecargadas con responsabilidades de coordinación.

Creator

El patrón Creator se aplica cuando una clase es responsable de crear objetos que conoce o que utiliza directamente.

En el sistema, ReservaGestor puede actuar como creador de una reserva porque reúne o coordina los datos necesarios para construirla, como el huésped, la habitación, las fechas y la estrategia de descuento. También EstadiaGestor puede crear un pago al finalizar una estadía, ya que conoce el costo final que debe abonarse.

Esto permite que la creación de objetos quede ubicada en clases que tienen sentido dentro del flujo del sistema.

Low Coupling

El patrón Low Coupling busca reducir la dependencia entre clases.

En Bragado, este principio se aplica porque muchas clases dependen de interfaces y no de clases concretas. Por ejemplo, Reserva no necesita conocer todas las estrategias de descuento existentes ni todos los canales de notificación. Solo trabaja con abstracciones como EstrategiaDescuento u ObservadorReserva.

Esto permite modificar una parte del sistema sin afectar innecesariamente a otras clases.

High Cohesion

El patrón High Cohesion indica que cada clase debe tener responsabilidades relacionadas entre sí.

En el sistema, las clases mantienen una función clara y coherente. Por ejemplo, Habitacion se encarga de sus datos, tipo, precio y estado; Huesped representa la información del huésped; Estadia administra el período de estadía y los costos; y los gestores agrupan operaciones relacionadas con reservas, habitaciones o estadías.

Esto mejora la comprensión del código y facilita su mantenimiento.

Polymorphism

El patrón Polymorphism se aplica cuando distintas clases responden de manera diferente a una misma operación.

En el proyecto, este patrón aparece en varios lugares. En Strategy, cada estrategia calcula el descuento de una forma distinta. En Decorator, cada servicio adicional suma su propio costo a la estadía. En State, cada estado de reserva responde de manera diferente a operaciones como confirmar, cancelar o finalizar.

Gracias al polimorfismo, se evita el uso excesivo de condicionales y el sistema se vuelve más flexible y extensible.

12. Explicación de las clases principales

Habitacion (abstracta)

Representa una habitación con número, capacidad, precio por noche, tipo y estado. Define el comportamiento común y delega en sus subclases el precio y la descripción. Subclases: HabitaciónSimple (1 persona, \$50), HabitaciónDoble (2 personas, \$80), HabitaciónSuite (2 personas, \$150).

Reserva

Representa la reserva de una habitación por un huésped en un rango de fechas. Gestiona su ciclo de vida mediante un objeto EstadoReserva (State), calcula su costo aplicando una EstrategiaDescuento (Strategy) y notifica eventos a sus ObservadorReserva (Observer).

Estadia: Representa la estadía efectiva tras el check-in. Mantiene las fechas de check-in/check-out y un ComponenteEstadia (Decorator) con los servicios contratados. Calcula el costo total y genera el Pago en el check-out.

Huesped

Almacena los datos del huésped (nombre, email, teléfono) y su lista de reservas, permitiendo consultar su historial.

UsuarioInterno (abstracta)

Representa a los usuarios internos (Administrador, Recepcionista, Personal Administrativo) con su rol y validación de credenciales (login por email y contraseña).

Pago

Registra una transacción (id, monto, fecha, método) y genera el comprobante de pago.

Gestores (ReservaGestor, HabitacionGestor, EstadiaGestor)

Coordinan las operaciones del sistema (Controller de GRASP): crean y administran reservas, habitaciones y estadías, y validan los permisos según el rol del usuario.

13. Instrucciones de despliegue / ejecución del proyecto

Instrucciones para ejecutar el proyecto

1. Clonar o descargar el repositorio del proyecto:

git clone <https://github.com/juligali/Tp-GestionHoteleria>

2. Abrir el proyecto en un entorno de desarrollo compatible con Java (IntelliJ IDEA, Eclipse o Visual Studio Code).
3. Verificar que el proyecto tenga configurado el JDK correspondiente y la librería de JavaFX para la interfaz gráfica.
4. Ejecutar la clase principal según la forma de uso deseada:
 - Interfaz gráfica (JavaFX): ejecutar gui/GuiLauncher.java.
5. Para ejecutar la versión por consola, iniciar: Main.java.

Una vez iniciado, se pueden probar las principales funcionalidades: consulta de disponibilidad, creación y confirmación de reservas, check-in y check-out, aplicación de descuentos, gestión de servicios adicionales, pagos simulados, comprobantes y reportes.

14. Capturas de ejecución o ejemplos de uso

1. Pantalla de login (acceso interno / cliente).

Acceso al sistema

Bragado centraliza reservas, huéspedes, habitaciones y estadias en una experiencia simple para equipos de hotelería.

Accesos internos de prueba: admin/admin123 - recepcion/recep123 - administrativo/administ123

Usuario interno

Clave

Entrar como usuario interno

Cliente

Email del cliente

Entrar como cliente

2. Listado de habitaciones con sus estados.

Habitaciones disponibles

Administrador - ADMINISTRADOR

Selecciona un rango para ver las habitaciones sin reservas cruzadas en esas fechas.

21/6/2026

22/6/2026

Buscar disponibilidad

Numero	Tipo	Capacidad	Estado	Precio	Descripcion
101	SIMPLE	1	DISPONIBLE	\$50.0	Habitación simple para una persona
102	DOBLE	2	DISPONIBLE	\$80.0	Habitación doble para dos personas
201	SUITE	2	DISPONIBLE	\$150.0	Suite de lujo con servicios premium

3. Creación de una reserva (cliente o recepción).

Crear reserva

Administrador - ADMINISTRADOR

Nombre huesped

SIMPLE ▾

1

Email

2/6/2026

Telefono

17/6/2026

Crear reserva

ID	Huesped	Habitac...	Tipo	Ingreso	Egreso	Estado	Habitacio...	Amenities	Extra	Total actu...	Pago
1	Sin nombre	101	SIM...	2026-...	2026-...	PENDI...	\$750.0	Sin amenities c...	\$0.0	\$750.0	PEN...

4. Confirmación de reserva con descuento aplicado.

Confirmar / cancelar reservas

Administrador - ADMINISTRADOR

Temporada (15%)

Confirmar seleccionada

Cancelar seleccionada

ID	Huesped	Habitac...	Tipo	Ingreso	Egreso	Estado	Habitacio...	Amenities	Extra	Total actu...	Pago
1	Sin nombre	101	SIM...	2026-...	2026-...	CONFI...	\$637.5	Sin amenities c...	\$0.0	\$637.5	PEND...

5. Check-in y carga de servicios (desayuno/spa/estacionamiento).

Administrador - ADMINISTRADOR

Reserva	Huesped	Check-in	Check-out	Servicios	Total
1	Sin nombre	2026-06-20	null	15 noche(s) en Habitación simple para una persona	\$750.0

 Mensaje
 ✕



Check-in realizado para la reserva #1
 Ahora puedes agregar o quitar amenities desde esta pantalla.
 Total actualizado: \$637.5

Aceptar

6. Check-out con comprobante de pago generado.

Check-in / Check-out

Administrador - ADMINISTRADOR

Check-in reserva seleccionada

SPA

Agregar amenity

Quitar amenity

Efectivo

Check-out

Selecciona una reserva confirmada para check-in. Para agregar/quitar amenities o hacer check-out, selecciona una estadia de la tabla.

Reserva	Huesped	Check-in	Check-out		Total
1	Sin nombre	2026-06-20	2026-06-20	15 no	750.0

Mensaje

Check-out realizado

Reserva #1 finalizada.
Total habitacion con descuento: \$637.5
Cargo extra por amenities: \$0.0
Total final: \$637.5

=== COMPROBANTE DE PAGO ===
ID: 1
Monto: \$637.5
Fecha: 2026-06-20
Método: efectivo
=====

Aceptar

7. Vista de reportes (ocupación e ingresos).

Reportes

Administrador - ADMINISTRADOR

Habitaciones totales: 3

Disponibles: 1

Reservadas: 1

Ocupadas: 1

Ocupacion actual: 33.3%

Reservas activas: 2

Estadias registradas: 2

Amenity mas utilizado: Estacionamiento (2)

Pagos registrados: 1

Ingresos simulados: \$637.5

Actualizar reporte

15. Conclusión grupal

El desarrollo del trabajo nos permitió aplicar de forma integral los conceptos de la programación orientada a objetos y el diseño de software vistos durante la cursada. A lo largo del proyecto pudimos llevar un problema real - la gestión de un hotel - desde su análisis y definición de requisitos hasta una implementación funcional.

La incorporación de patrones de diseño (Builder, Factory Method, Decorator, State, Strategy y Observer) y de los principios SOLID y GRASP nos mostró en la práctica cómo estas herramientas mejoran la organización del código, reducen el acoplamiento y facilitan la extensión del sistema. Comprendimos que un buen diseño no se nota solo cuando el sistema funciona, sino sobre todo cuando hay que modificarlo o agregarle funcionalidades.

También identificamos limitaciones de la implementación actual - como la validación de disponibilidad por fechas o la gestión completa de usuarios - que dejamos documentadas como mejoras a futuro. Reconocer estas limitaciones forma parte del aprendizaje: nos permite ser conscientes del alcance real de lo construido y de los pasos necesarios para convertirlo en un sistema productivo.

En conjunto, el trabajo nos dejó una base sólida tanto en lo técnico como en lo grupal, reforzando la importancia de la planificación, la división de tareas y las buenas prácticas de diseño.

16. Reflexión individual de cada integrante

Luz Barrientos

Mi mayor aporte estuvo en la base estructural del proyecto: definí los paquetes iniciales, implementé las clases del modelo de habitaciones y de usuarios, los enums y las factories, y participé en la versión inicial de los gestores. También trabajé bastante en la interfaz gráfica y en el cierre del estilo CSS. Lo que más me costó al principio fue organizar la estructura de paquetes de forma que cada responsabilidad quedara separada, pero entender el patrón Factory Method para la creación de habitaciones me ayudó a ver el valor de un diseño extensible: agregar un nuevo tipo de habitación no implica tocar el resto del sistema. Si tuviera que mejorar algo, profundizaría en la lógica de disponibilidad para que considere fechas y capacidad, que quedó como una de nuestras mejoras a futuro.

Juliana Galiano

Me enfoqué principalmente en los patrones de comportamiento: implementé el patrón State para el ciclo de vida de la reserva, el patrón Observer para las notificaciones y el patrón Strategy para los descuentos, además de la clase Pago. También participé en la configuración inicial del proyecto, en el desarrollo de la GUI y en la redacción del README. Lo que más me aportó fue trabajar con el patrón State, porque me hizo entender cómo reemplazar largas

cadenas de condicionales por clases que encapsulan el comportamiento de cada estado, dejando la clase Reserva mucho más limpia. La mayor dificultad fue coordinar los patrones entre sí (que el cambio de estado dispare las notificaciones, por ejemplo). Como mejora, me gustaría sumar nuevos canales de notificación y recordatorios automáticos, aprovechando que el Observer ya deja esa puerta abierta.

Paloma Macchión

Mi trabajo se centró en la clase Reserva y su Builder, la clase Estadia, el patrón Decorator para los servicios adicionales y, sobre todo, la clase Main y la integración de todos los patrones desde la consola. También colaboré en la GUI y en los ajustes finales del proyecto. Lo que más me costó fue integrar todas las piezas que cada uno había desarrollado por separado y lograr que el flujo completo (reservar, confirmar, check-in, agregar servicios, check-out y pago) funcionara de forma coherente. El patrón Decorator fue el que más me sirvió para entender la composición dinámica: poder combinar servicios sin crear una clase por cada combinación. En la etapa final también participé en un refactor del patrón State para sacar la lógica de presentación del modelo. Si tuviera que seguir mejorando el sistema, trabajaría en la persistencia de datos para que la información no se pierda al cerrar la aplicación.